Bases de Données Fall, 2026

TP 5 (velo)

This TP has five steps.

Important: Take your time to read and understand what to do.

It is not difficult . . . *as long as you take time to read and understand the subject.*

In this TP, we will work with the Accidents Velo data.

The entire data is in two files—`accidentsVelo.csv` and `data-description.json`.

It contains a lot of information about bike accidents that have happened in France—latitude/longitude coordinates, gender, time of accident, type of accident, . . .

The entire data is from here:

France Bike Accidents Data:
`https://www.data.gouv.fr/datasets/accidents-de-velo/`

USING USPN LABS DATABASE

- Type the following command in your `linux` account to start the `postgreSQL prompt`:

  ```
  psql -h postgres -U m1info_xx
  ```

  where `xx` is your account number between 1 and 90. The password is `M1INFO_xx`, with the same value of `xx` as your account number.

I now summarize the five steps below, and then starting next page, each step is explained in more detail.

TP Goals

1. **Download and understand the `accidentsVelo.csv` file.**

   Download the file `accidentsVelo.csv` and examine it *carefully* to understand what kind of data it contains. Each line is information about one accident.

2. **Download and understand the `data-description.json` file.**

   Download the file `data-description.json` and examine it *carefully* to understand what kind of data it contains. It contains, in the `.json` format, information about each attribute in the file `accidentsVelo.csv`.

   Look at both `accidentsVelo.csv` and `data-description.json` to make sure you understand the meaning of each attribute in `accidentsVelo.csv`.

3. **Write a first Python script (`myparseJSON.py`) to process `data-description.json`.**

   Next, write a Python script that will read `data-description.json` and output the commands to do several CREATE TABLEs. The script will also output INSERT INTO ... commands to insert data into these new tables.

4. **Write a second Python script (`myparseCSV.py`) to process `accidentsVelo.csv`.**

   Next, write Python script to read `accidentsVelo.csv` line by line, and for each line, the code will output a SQL INSERT VALUES command to insert that line into the table accidentsvelo you created in `Step 3`. So the number of SQL INSERT VALUES commands will be exactly equal to the number of rows in the input `.CSV` file.

5. **Solve new queries using the tables created.**

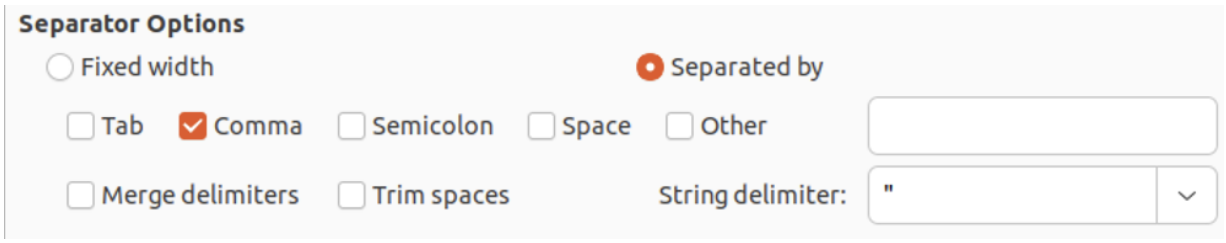
   Finally, once you have created all the tables from the `.json` and the `.CSV` files, and entered all the data into them, solve all the SQL query problems that are given.

1 Download and understand the `accidentsVelo.csv` file.

Download the file `accidentsVelo.csv`.

Then, open the file with a text editor, or even better, with `soffice` (also called libreoffice).

If you open with a spreadsheet editor (e.g., libreoffice), don't forget to put ',' as the `ONLY` separator between attributes when you open/import the file. It should be like this:



It contains the list of all bike accidents in France.

There are 80023 **rows in this file.**

Here are the list of attributes in this file:

```
Num_Acc,date,an,mois,jour,hrmn,
dep,com,lat,long,agg,int,
col,lum,atm,catr,circ,nbv,
prof,plan,lartpc,larrout,surf,infra,
situ,grav,sexe,age,trajet,secuexist,
equipement,obs,obsm,choc,manv,vehiculeid,
typevehicules,manoeuvehicules,numVehicules
```

For example, here is the first data line, containing information about one bike accident that happened:

```
200500000030,2005-01-13,2005,janvier,jeudi,19:45,
62,62331,50.3,2.84,2,1,
3,5,1,3,2,00,
-1,1,000,050,1,0,
1,4,1,58,5,3,
0,00,2,8,11,200500000030B02,
18,17,1
```

2 Download and understand the `data-description.json` file.

a) To understand the values of attributes in the `accidentsVelo.csv`, we have to look at the file `data-description.json`.

b) You can open this file in firefox:

```
firefox data-description.json
```

You can see that it contains, for each attribute in the `accidentsVelo.csv`, a table which gives the description of the value of each attribute.

c) This file is just a text file with a special format. You can open it in a text editor:

```
kate data-description.json
```

Examine the file and try to understand its format.

3 Write a first Python script to read and process `data-description.json`.

a) I have given you a starting Python script called `myparseJSON.py`. It contains code that will read and print all information in the file `data-description.json`.

myparseJSON.py

```
import json

with open('data-description.json') as f:
    i = 0
    for name, info in json.load(f)['properties'].items():
        attrib_name = str(name)
        attrib_domain = info['type']
        #get the dict of x-labels. Return empty dict if x-labels not found.
        attribute_dict = info.get('x-labels',{})

        print("Information on attribute ", i)
        print ("\t", name)
        print ("\t", info['type'])
        print(attribute_dict)

        for (x,y) in attribute_dict.items():
            print ("\t\t", x, y)

        i = i + 1
        print("
    ------------------------------------------------------------------------")
```

b) Put `data-description.json` in the same directory, and run it with:

python3 myparseJSON.py

Understand carefully what it prints, and the code.

c) Now, **modify `myparseJSON.py` so that it prints:**

(a) One CREATE TABLE command to create the main table `accidentsvelo`.
Note: for the attribute `date`, use the domain type `date` instead of `text`.

(b) For *each* attribute in table `accidentsvelo` for which there are values with descriptions in the `data-description.json`, it should print a CREATE TABLE command.

(c) Then, it should also print the INSERT INTO ... commands to insert the value and its corresponding description in the table.

d) This is what it should print (I only give the first few tables):

```
create table accidentsVelo (
 Num_Acc TEXT,
 date DATE,
 an NUMERIC(5,0),
 mois TEXT,
 jour TEXT,
 hrmn TEXT,
 dep TEXT,
 com TEXT,
```

```
 lat REAL,
 long REAL,
 agg NUMERIC(5,0),
 int NUMERIC(5,0),
 col NUMERIC(5,0),
 lum NUMERIC(5,0),
 atm NUMERIC(5,0),
 catr NUMERIC(5,0),
 circ NUMERIC(5,0),
 nbv TEXT,
 prof NUMERIC(5,0),
 plan NUMERIC(5,0),
 lartpc REAL,
 larrout REAL,
 surf NUMERIC(5,0),
 infra NUMERIC(5,0),
 situ NUMERIC(5,0),
 grav TEXT,
 sexe NUMERIC(5,0),
 age NUMERIC(5,0),
 trajet NUMERIC(5,0),
 secuexist NUMERIC(5,0),
 equipement TEXT,
 obs NUMERIC(5,0),
 obsm NUMERIC(5,0),
 choc NUMERIC(5,0),
 manv NUMERIC(5,0),
 vehiculeid TEXT,
 typevehicules TEXT,
 manoeuvehicules TEXT,
 numVehicules NUMERIC(5,0)
);

create table t_mois(
  mois TEXT,
 description TEXT
);
INSERT INTO t_mois VALUES ('janvier', '01-Janvier');
INSERT INTO t_mois VALUES ('février', '02-Février');
INSERT INTO t_mois VALUES ('mars', '03-Mars');
INSERT INTO t_mois VALUES ('avril', '04-Avril');
INSERT INTO t_mois VALUES ('mai', '05-Mai');
INSERT INTO t_mois VALUES ('juin', '06-Juin');
INSERT INTO t_mois VALUES ('juillet', '07-Juillet');
INSERT INTO t_mois VALUES ('août', '08-Août');
INSERT INTO t_mois VALUES ('septembre', '09-Septembre');
INSERT INTO t_mois VALUES ('octobre', '10-Octobre');
INSERT INTO t_mois VALUES ('novembre', '11-Novembre');
INSERT INTO t_mois VALUES ('décembre', '12-Décembre');
```

```
create table t_agg(
  agg NUMERIC(5,0),
 description TEXT
);
INSERT INTO t_agg VALUES ('1', 'Hors agglomération');
INSERT INTO t_agg VALUES ('2', 'En agglomération');

create table t_int(
  int NUMERIC(5,0),
 description TEXT
);
INSERT INTO t_int VALUES ('0', 'Non renseigné');
INSERT INTO t_int VALUES ('1', 'Hors intersection');
INSERT INTO t_int VALUES ('2', 'Intersection en X');
INSERT INTO t_int VALUES ('3', 'Intersection en T');
INSERT INTO t_int VALUES ('4', 'Intersection en Y');
INSERT INTO t_int VALUES ('5', 'Intersection à plus de 4 branches');
INSERT INTO t_int VALUES ('6', 'Giratoire');
INSERT INTO t_int VALUES ('7', 'Place');
INSERT INTO t_int VALUES ('8', 'Passage à niveau');
```

e) Run this `Python` script with

python3 `myparseJSON.py`> schema.sql

This should output all the insert lines into the file `schema.sql`.

f) Log into `psql` with your account name and create the schema by running the command :

```
\i schema.sql
```

After this, the table accidentsvelo should be created, as well as all the tables and their data for the attribute descriptions.

You should see this:

```
velodb=# \d
           List of relations
 Schema |     Name      | Type
--------+---------------+-------
 public | accidentsvelo | table
 public | t_agg         | table
 public | t_atm         | table
 public | t_catr        | table
 public | t_choc        | table
 public | t_circ        | table
 public | t_col         | table
 public | t_grav        | table
 public | t_infra       | table
 public | t_int         | table
 public | t_larrout     | table
 public | t_lartpc      | table
 public | t_lum         | table
 public | t_manv        | table
 public | t_mois        | table
 public | t_nbv         | table
 public | t_obs         | table
 public | t_obsm        | table
 public | t_plan        | table
 public | t_prof        | table
 public | t_secuexist   | table
 public | t_sexe        | table
 public | t_situ        | table
 public | t_surf        | table
 public | t_trajet      | table
(25 rows)
```

4 Write a second Python script to read and process accidentsVelo.csv.

a) Use Python to read the file accidentsVelo.csv and print INSERT INTO accidentsvelo line for each line.

For example, the first line should be :

```
INSERT INTO accidentsVelo VALUES( '200500000030','2005-01-13','2005','janvier','jeudi',
    '19:45','62','62331','50.3','2.84','2','1','3','5','1','3','2','00','-1','1','000','050',
    '1','0','1','4','1','58','5','3','0','00','2','8','11','200500000030B02','18','17','1');
```

b) I have given you the following starting Python file called myparseCSV.py, which *already* reads the input file, breaks it into lines, and breaks each line into attributes that are separated by the ';' symbol.

You should run this file first, with

python3 myparseCSV.py

Examine what it prints, and understand the Python code in it.

myparseCSV.py

```
import re

#open a file for read-only purposes
f = open("accidentsVelo.csv", 'r')

labels = f.readline().rstrip("\n").split(",")

#loop over each line of the input file f
for line in f:
    items = line.rstrip("\n").split(",")

    #num_attributes = number of attributes in this line
    num_attributes = len(items)

    #loop over each attribute in the array items
    i = 0

    while i < num_attributes:
        #replace ' with '', for strings in postgreSQL
        item = items[i].replace("'", "''")

        print("Attribute ", i, "is: ", item)
        i = i + 1
    print("-------------------------------------------------")
```

I give this file to you to get started and it contains **all** the necessary 'ingredients' to do this TP. Once you have understood the code, *modify it as you want* so that it prints the required

```
INSERT INTO accidentsvelo VALUES (...)
```

commands.

c) Run this Python script with

python3 myparseCSV.py> data.sql

This should output all the insert lines into the file data.sql.

d) Log into psql with your account name and add in the data by running the command :

```
\i data.sql
```

After this, the entire data from the file `accidentsVelo.csv` should be in the table accidentsvelo.

e) To check if data is entered correctly, you can try some queries. For example:

```
velodb=# SELECT count(*) FROM accidentsvelo WHERE age >= 60;
 count
-------
 15632
(1 row)
```

5 Solve new queries using the tables created.

Now write the `SQL` code, and then run to verify, the following queries. The table after each question gives the result that you should get.

a.) Give the area code of the three cities with the most number of accidents.

com	*num*
67482	1394
33063	1174
75112	1086

b.) For the accidents that happen in "Piste Cyclable", give the percentages where the bicyclist was unharmed, slightly hurt, hospitalised, and dead.

description	*?column?*
Blessé hospitalisé	19.5356917245735944
Tué	1.5476942514213519
Indemne	15.0979153506001263
Blessé léger	63.8186986734049274